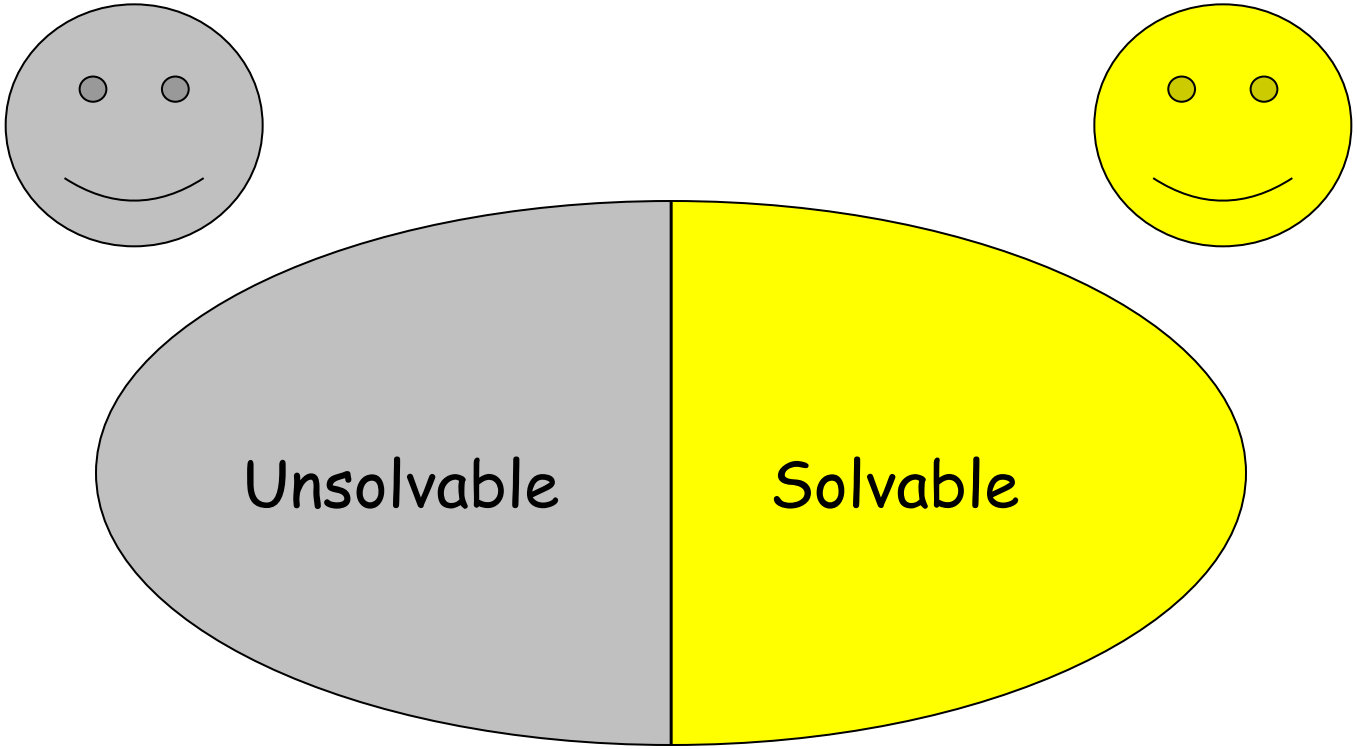
Outline

- Decision and Optimization Problems
- Polynomial time algorithms (Tractable)
- Decidable vs. undecidable problems
- Class P and NP
- Polynomial-Time Reducibility
- NP-Hardness and NP-Completeness
- Examples: TSP, Circuit-SAT, Knapsack
- Proving NP-completeness

Unsolvability vs. Solvability

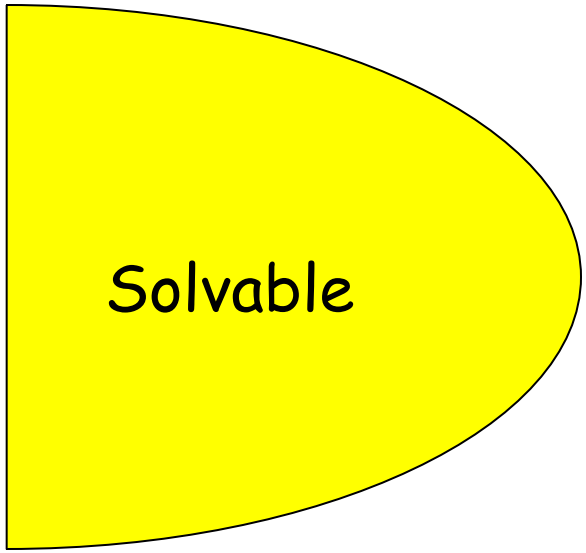


Complexities

Can a solvable problem be solved in practical sense?

Time Complexity

Space Complexity



Decision and Optimization Problems

- Decision Problem: computational problem with intended output of “yes” or “no”, 1 or 0
    - e.g. Is there a solution better than some given bound?
- Optimization Problem: computational problem where we try to maximize or minimize some value
- Introduce parameter k and ask if the optimal value for the problem is a most or at least k. Turn optimization into decision

Polynomial-time Alg., Tractable

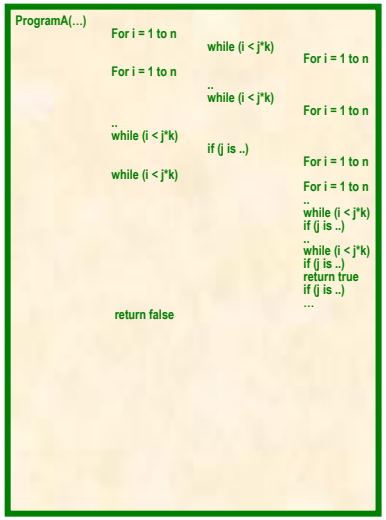
All algorithms we have studies so far like searching, sorting, finding MSTs, finding shortest paths, etc in general takes $O(n^k)$ time where k is a constant. These are called **Polynomial-time algorithms. So, let's generalize these all to P class.**

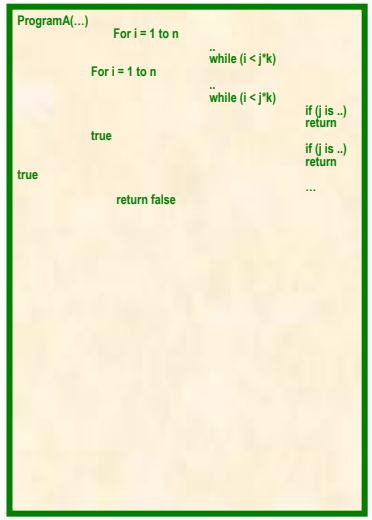
- class P : class of all problems that can be solved by some algorithms that takes polynomial time.
  // there exist a deterministic algorithm

- Now, the question can be asked is that "whether there are problems that can't be solved in polynomial time?"

Why study polynomial-time algorithms

From experience, once a polynomial-time algorithm for a problem is discovered (eg. $\Theta(n^{100})$), more efficient algorithm often follow.



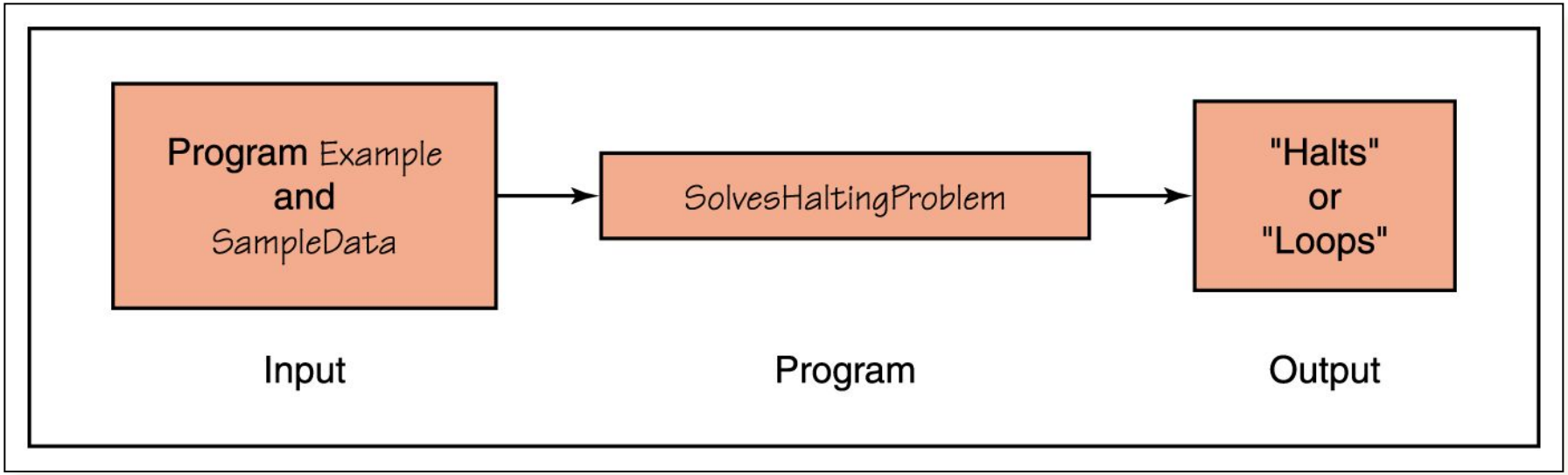


A problem that can be solved in polynomial time in one model (one computer architecture) can be solved in polynomial time in another.

Decidable vs undecidable problems

- If you have learned TOC, then you must be knowing that there are some problems, those even cant be solved at all. Such problems are called **undecidable problems.** For ex: halting problem, there is no algorithm exist for it.
- There are some problems that cannot be solved in polynomial-time. Eg. **Turing's famous Halting Problem** cannot be solved by any computer, no matter how much time is given:

Halting Problem: The problem of determining in advance whether a particular program or algorithm will terminate (will not run forever).

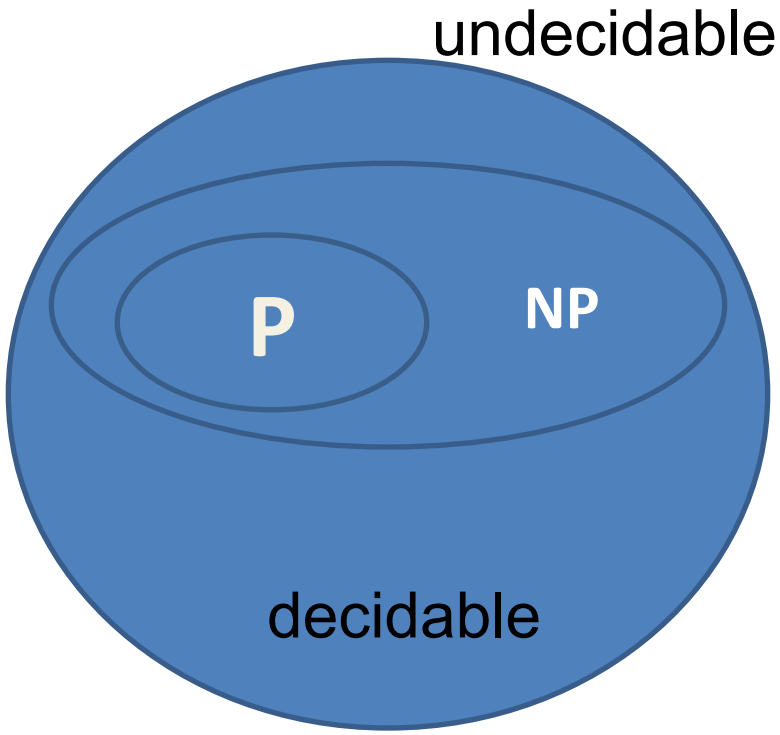


Decidable vs undecidable problems

- Now lets look at the class of problems that are decidable.
- Question is : “are there algorithms that can be **always solvable in polynomial time?**”
- Answer is : NO. There are problems that require more than polynomial time. For Ex: In TOC, CFL is a regular language or not. This problem is decidable problem. But, it is sure that there is no polynomial time algorithm to solve it.

NP

- So, there are algorithms that takes more than polynomial time.
- Now, there is another class of problem that have a solutions in this class, called NP which turns out to be very interesting.
- So, NP is a class of problem that can be solved by algorithms that are nondeterministic and work in polynomial time.



NP

- NP class : class of all problems that can be solved by nondeterministic algorithm that takes polynomial time.

Deterministic

```
If(x<5)
   {  ……..
   }
else
   { …..
   }
```

Nondeterministic

```
if(*)
        {  ……..
}
else
     {  …
}
```

Polynomial-time verification

- A verification algorithm is a two-argument algorithm A, where one argument is an input binary string x and the other is a binary string y called a certificate.
- A two-argument algorithm A verifies an input string x if there is a certificate y such that

  $A(x, y) = 1$.

3 classes of problems: P, NP, NPC

Class P: Solvable in Polynomial-time ($O(n^k)$) for some constant k.

Class NP: "Verifiable" in polynomial-time.

- Given a "certificate" of a solution, we can verify that the certificate is correct in $O(n^k)$ time.
- NP is the class of decision problems for which there is a polynomially bounded non-deterministic algorithm.
- The name NP comes from "Non-deterministic Polynomially bounded." // there exist a non-deterministic algorithm.

Hamiltonian-cycle problem: Find a simple cycle that contains each vertex of a given directed graph.
A certificate would be a path of n vertices.
It is easy to check that this certificate is correct, in $O(n^k)$ time.

A problem solvable in polynomial-time must be verifiable in polynomial time. i.e. $P \subseteq NP$.

Class NPC (The class of NP-complete problems): These problems are in NP and are as "hard" as any problem in NP, *eg. Hamiltonian-cycle*

P, NP, NPC

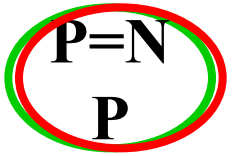
Solvable in Polynomial-time

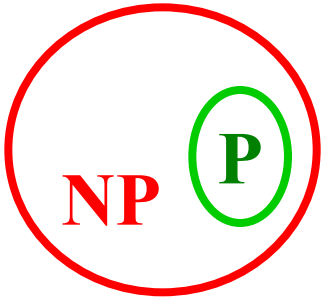
Verifiable in polynomial-time

We know P ⊆ NP

We are interested in whether P = NP ? --- (*)

If we can find a polynomial-verifiable problem that is proved not polynomial-time solvable, then we can disprove (*).
But so far we cannot find any.





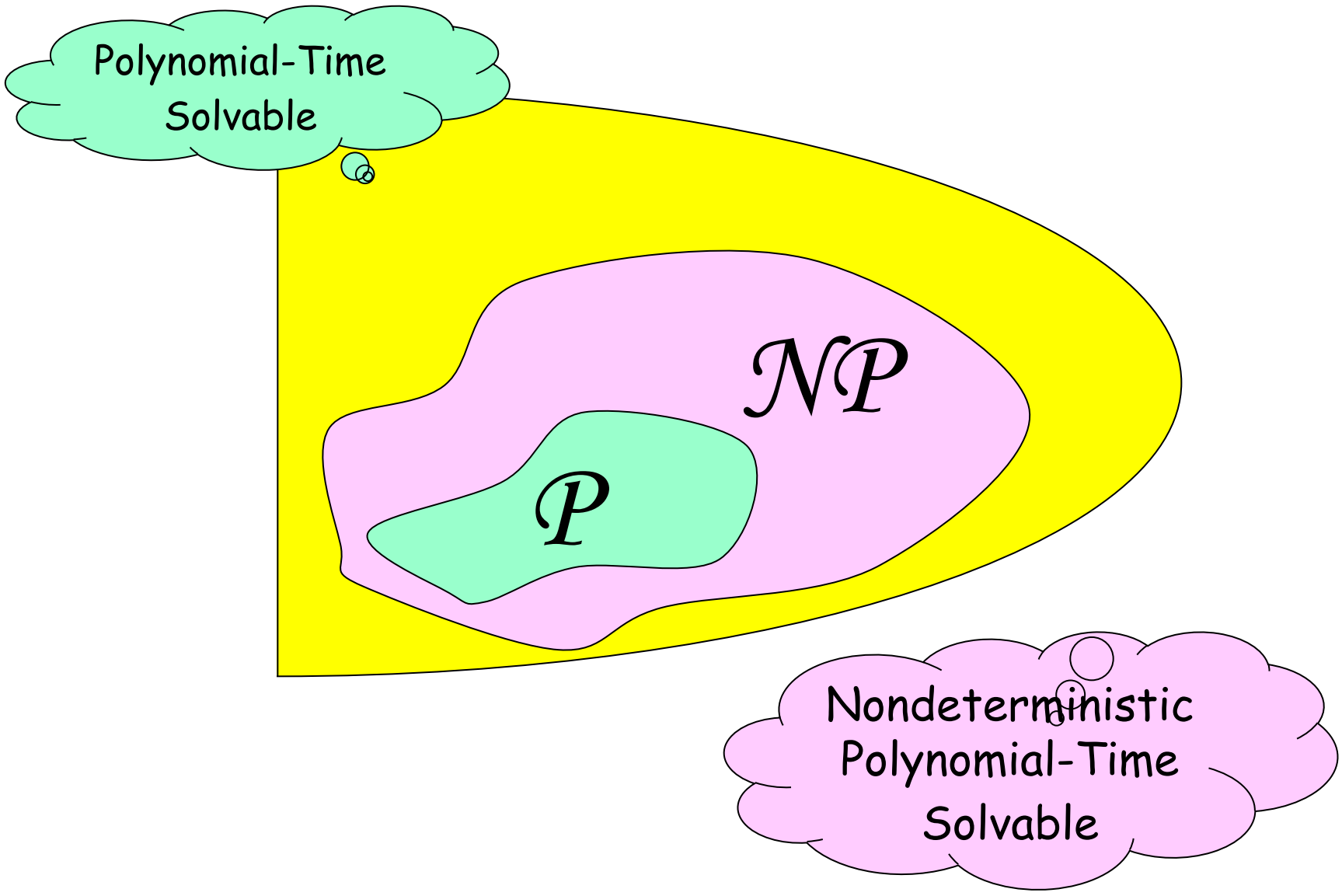
Class NPC (The class of NP-complete problems):

There are many problems in NPC. So it is important to know whether these problems are tractable.

Moreover, if a polynomial time solution can be found for any one of them, then all NP problems are polynomial-time solvable (hence * can be proved).

However, so far no such solution is discovered.

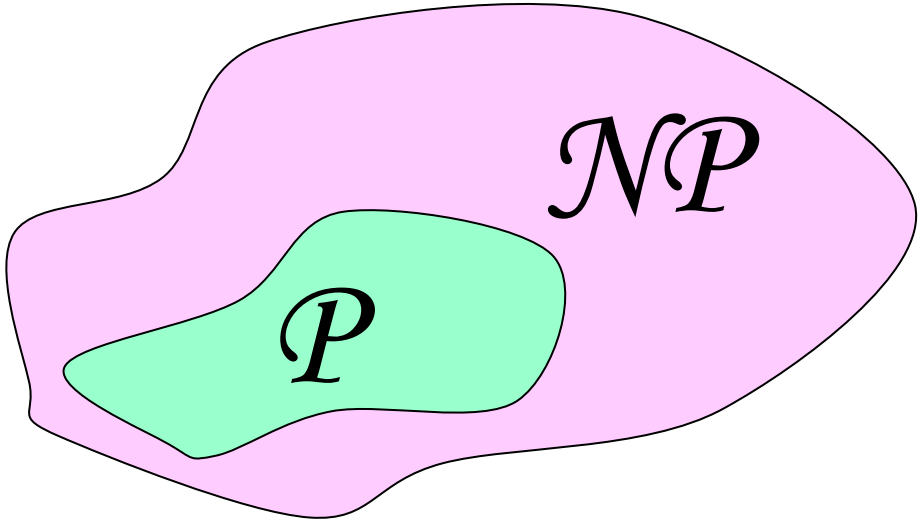
$\mathcal{P}$ and $\mathcal{NP}$



$\mathcal{P}$ and $\mathcal{NP}$

$\mathcal{P} = \mathcal{NP}$?

No answer, now.

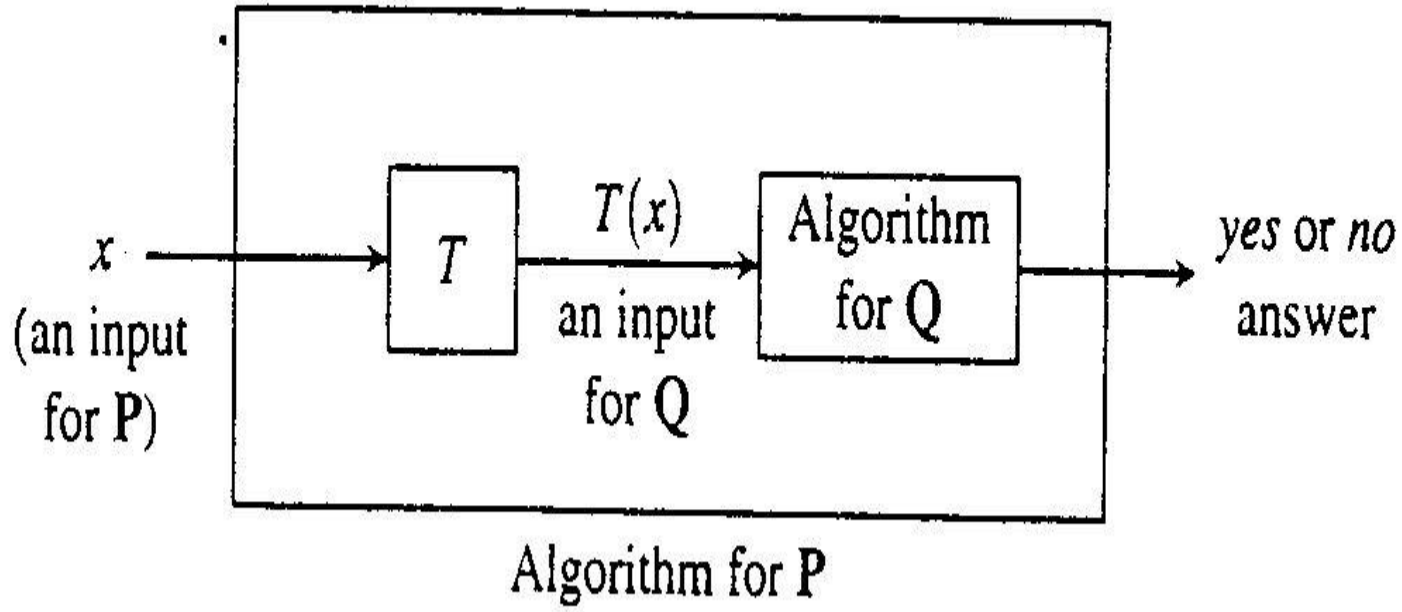


The Class NP-Hard and NP-Complete

- A problem Q is *NP-complete*
  - if it is in NP and
  - it is NP-hard.
- A problem Q is *NP-hard*
  - if *every* problem in NP
  - is reducible to Q.
- A problem P is ***reducible*** to a problem Q if
  - there exists a **polynomial reduction function T** such that
    - For every string x,
    - if x is a yes input for P, then T(x) is a yes input for Q
    - if x is a no input for P, then T(x) is a no input for Q.
    - T can be computed in polynomially bounded time.

Polynomial Reductions

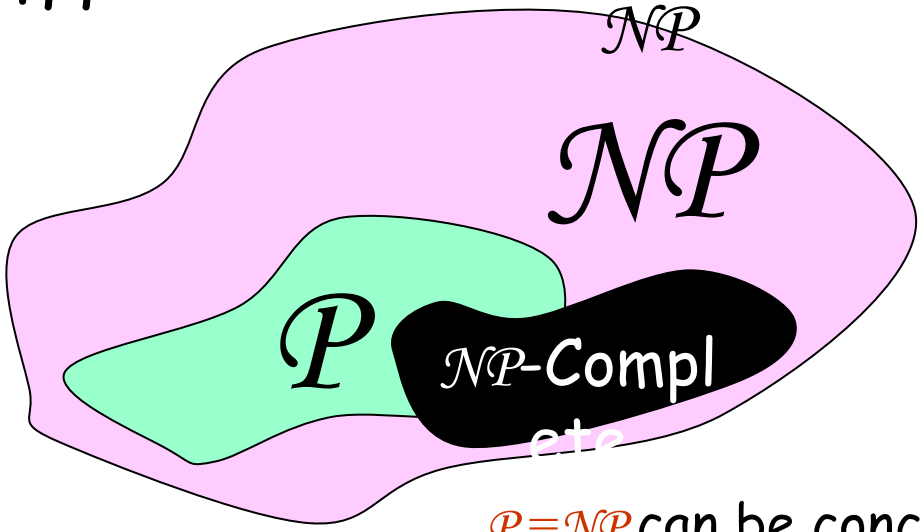
- Problem P is reducible to Q
  - P ≤p Q
  - Transforming inputs of P
    - to inputs of Q
- Reducibility relation is transitive.



$\mathcal{NP}$-Completeness

$L \in \mathcal{NP}$-Complete iff

1. $L \in \mathcal{NP}$
2. $\mathcal{NP} \in \mathcal{NP}$ $\xrightarrow{p.r}$ L



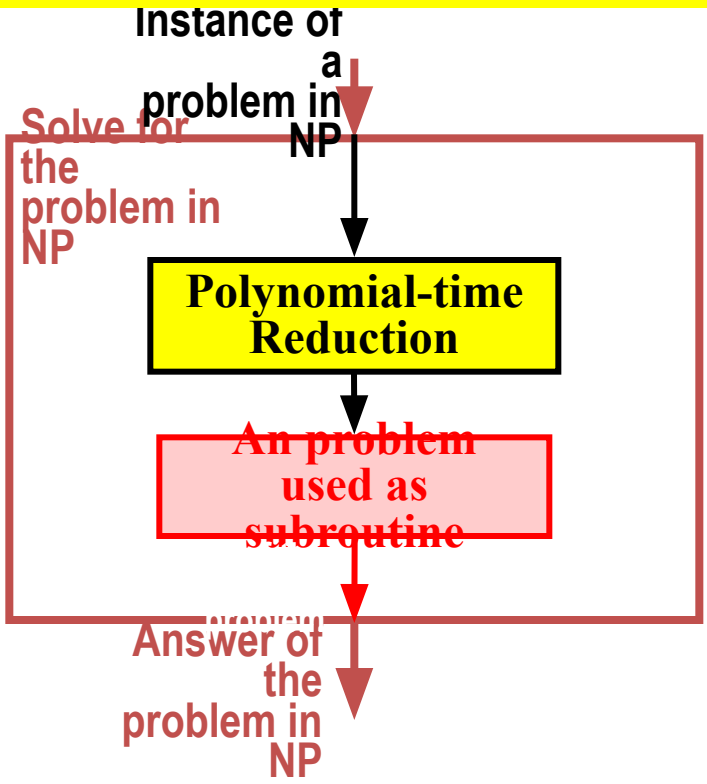
$\mathcal{P}=\mathcal{NP}$ can be concluded if you can find one $L \in \mathcal{NP}$-Complete being polynomial-time solvable, i.e., $L \in \mathcal{P}$.

How to prove NP-Completeness

Note that, if we can solve a problem P_1 by mainly solving P_2, this does not imply P_1 is harder than P_2. Indeed, we can say the P_2 algorithm is a powerful engine that can solve for P_1, although maybe P_1 is only a piece of cake.

NP-complete problems are the hardest ones of all NP problems.

One method to prove that a particular problem is NP-complete:



More examples

1. Shortest vs longest simple paths

   Single-source shortest paths can be found in O(VE) time.

   But finding longest simple path between 2 vertices is NP-complete

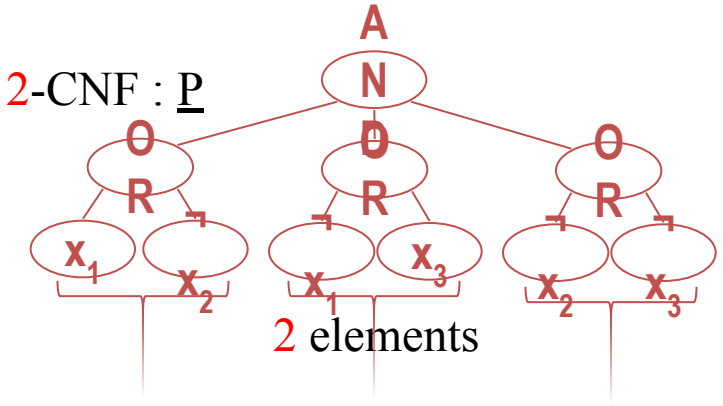
2. Euler tour vs hamiltonian cycle

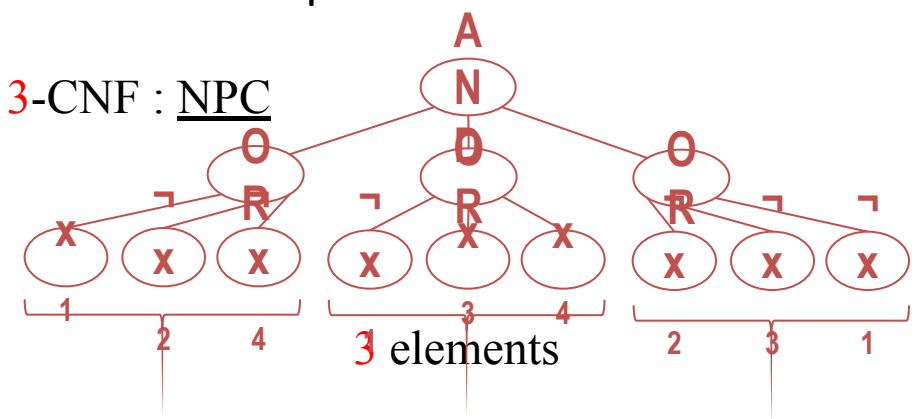
   Euler tour: traverses **each edge** of a connected, directed graph exactly once (O(E))

   Hamiltonian cycle: a simple cycle that contains **each vertex** of a given graph: NP-complete

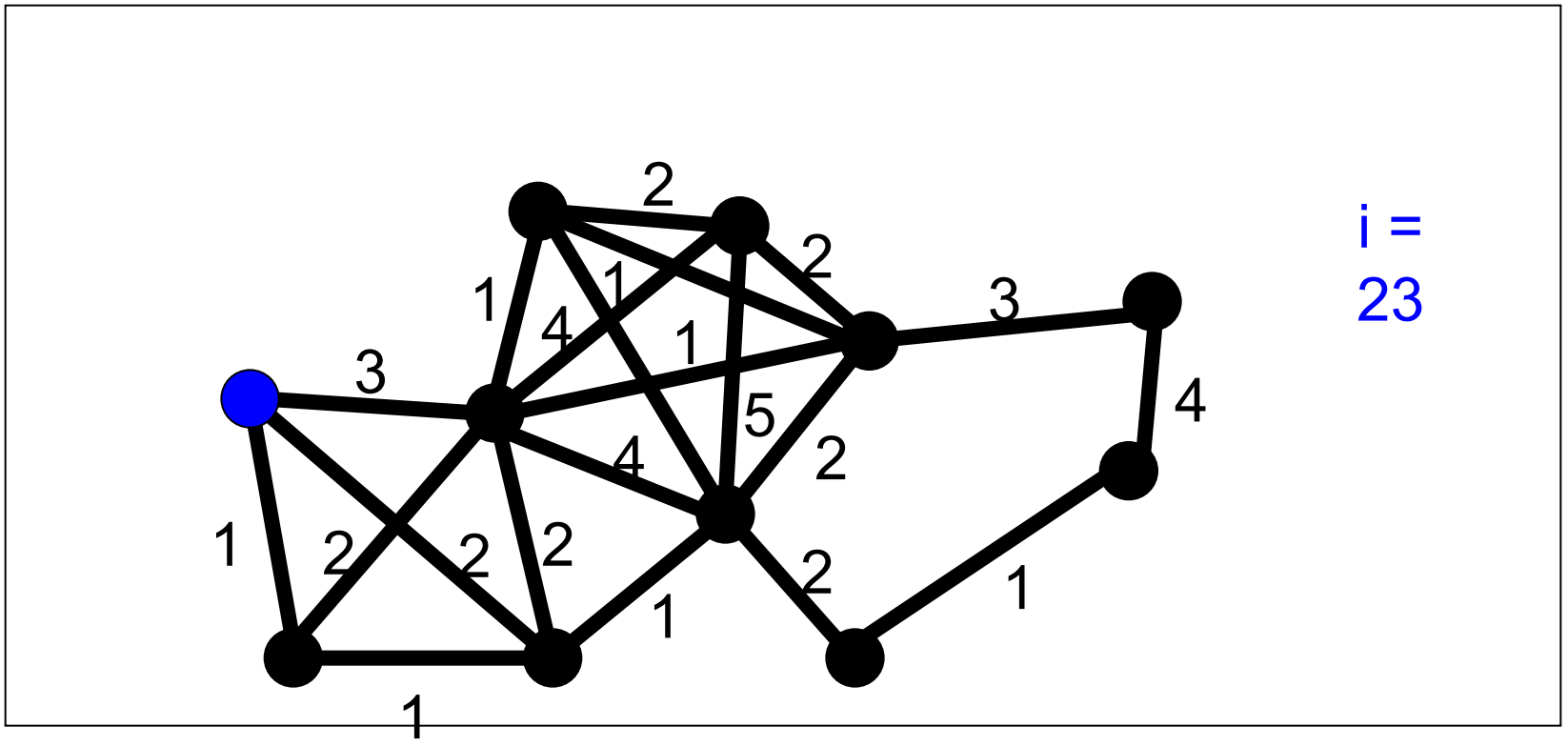
3. 2-CNF satisfiability vs 3-CNF satisfiability (k-conjunctive normal form)

   A boolean formula is satisfiable if there is some assignment of the 0/1 variable values that makes the formula results in "1". Example:



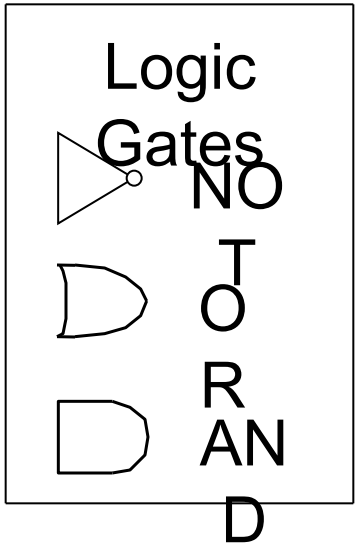


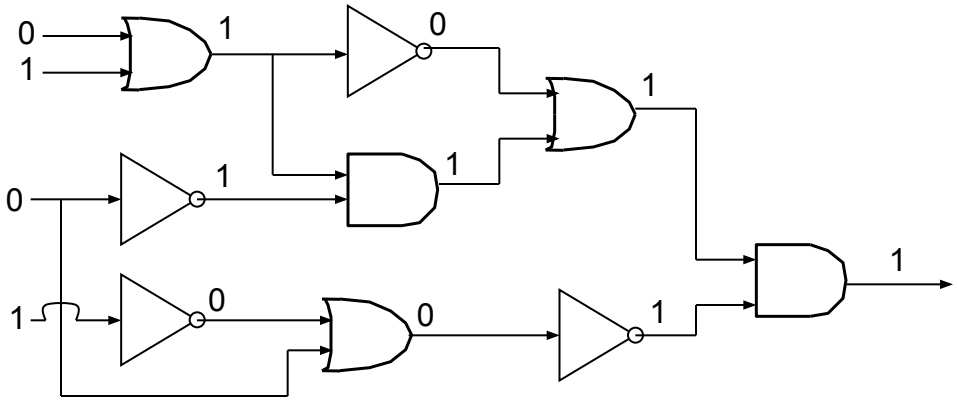
TSP



- For each two cities, an integer cost is given to travel from one of the two cities to the other. The salesperson wants to make a minimum cost circuit visiting each city exactly once.

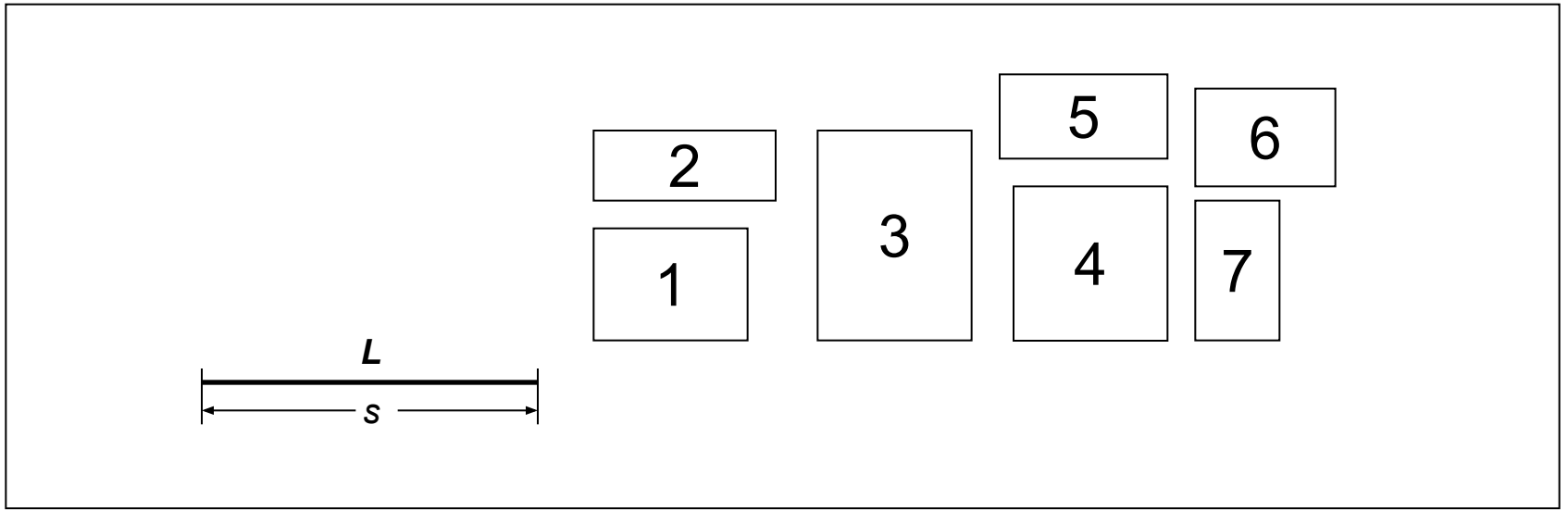
Circuit-SAT





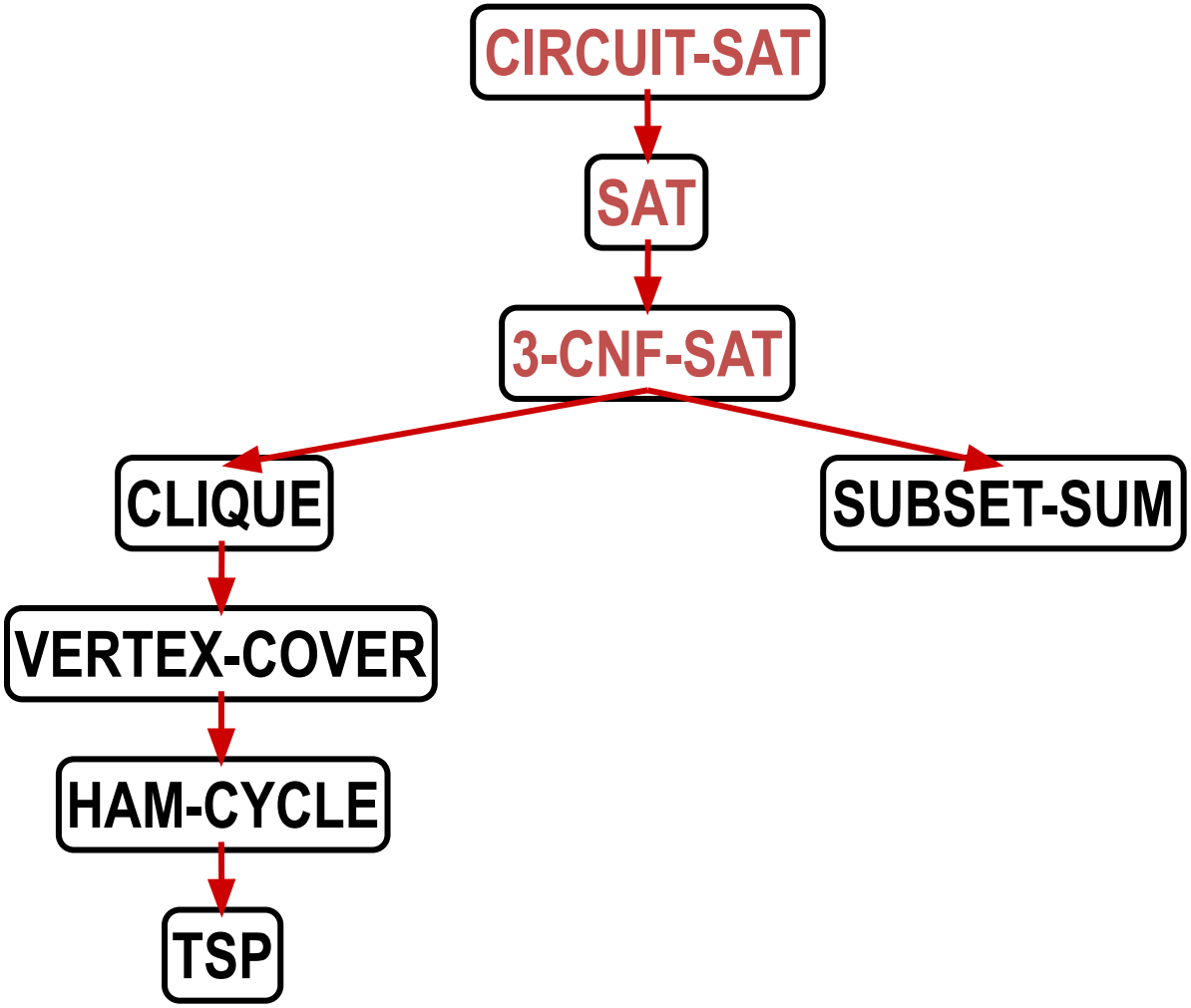
- Take a Boolean circuit with a single output node and ask whether there is an assignment of values to the circuit's inputs so that the output is "1"

Knapsack



- Given *s* and *w* can we translate a subset of rectangles to have their bottom edges on L so that the total area of the rectangles touching L is at least *w*?

NP-Complete problems



Review: NP-Completeness

- *What do we mean when we say a problem is in P?*
  - A solution can be found in polynomial time.
- *What do we mean when we say a problem is in **NP**?*
  - A solution can be verified in polynomial time.
- *What is the relation between P and NP?*
  - P $\subseteq$ NP, but no one knows whether P = NP.
- *What does it mean if we can reduce problem P to problem Q?*
  - P is "no harder than" Q.
- *How do we reduce P to Q?*
  - Transform instances of P to instances of Q in polynomial time s.t. Q: "yes" iff P: "yes"
- *What does it mean if Q is NP-Complete?*
  - Q is NP-Hard and Q $\in$ **NP.**
- *How do we usually prove that a problem R is NP-Complete?*

  - Show R $\in$NP, and reduce a known NP-Complete problem Q to R.